

Przekształcanie danych z tidyr i dplyr

slajdy pomocnicze do szkolenia IDUB

Czego się nauczymy w tym module?

- przekształcania danych szerokich (wide) na długie (long)
- wybierania kolumn funkcją select
- wybierania wierszy funkcją filter
- grupowania i sumowania funkcjami group_by i summarise
- filtrowania wartości brakujących
- tworzenia nowych zmiennych funkcją mutate()
- łączenia ramek danych funkcjami *_join

Przekształcanie danych długich na szerokie

- funkcja `pivot_longer()` pakietu `tidyr` przekształca dane szerokie na długie

```
podatki_long <- pivot_longer(podatki,  
  cols = c(2:5),  
  names_to = "kraj",  
  values_to = "podatek")
```

	rok	USA	WB	Niemcy	Francja
1	1900	0.0000	0.00000000	0.03	0.000000
2	1901	0.0000	0.00000000	0.03	0.000000
3	1902	0.0000	0.00000000	0.03	0.000000
4	1903	0.0000	0.00000000	0.03	0.000000
5	1904	0.0000	0.00000000	0.03	0.000000
6	1905	0.0000	0.00000000	0.03	0.000000
7	1906	0.0000	0.00000000	0.03	0.000000
8	1907	0.0000	0.00000000	0.03	0.000000

Showing 1 to 8 of 114 entries, 5 total columns

	rok	kraj	podatek
1	1900	USA	0.00000000
2	1900	WB	0.00000000
3	1900	Niemcy	0.03000000
4	1900	Francja	0.00000000
5	1901	USA	0.00000000

Showing 1 to 5 of 456 entries, 3 total columns

Różnica między danymi długimi a szerokimi w formacie długim każda zmienna ma jedną kolumnę

cztery wartości zmiennej
kraj mają własne kolumny



	rok	USA	WB	Niemcy	Francja
1	1900	0.0000	0.00000000	0.03	0.00000
2	1901	0.0000	0.00000000	0.03	0.00000
3	1902	0.0000	0.00000000	0.03	0.00000
4	1903	0.0000	0.00000000	0.03	0.00000
5	1904	0.0000	0.00000000	0.03	0.00000
6	1905	0.0000	0.00000000	0.03	0.00000
7	1906	0.0000	0.00000000	0.03	0.00000
8	1907	0.0000	0.00000000	0.03	0.00000

Showing 1 to 8 of 114 entries, 5 total columns

zmienna kraj w jednej kolumnie



	rok	kraj	podatek
1	1900	USA	0.00000000
2	1900	WB	0.00000000
3	1900	Niemcy	0.03000000
4	1900	Francja	0.00000000
5	1901	USA	0.00000000
6	1901	WB	0.00000000
7	1901	Niemcy	0.03000000
8	1901	Francja	0.00000000

Showing 1 to 8 of 456 entries, 3 total columns

układ długi jest wygodniejszy dla
prowadzenia analiz i tworzenia wykresów

Przekształcanie danych szerokich na długie

- funkcja `pivot_wider()` pakietu `tidyr` przekształca dane szerokie na długie

```
podatki_wide <- pivot_wider(podatki_long,  
                             names_from = kraj,  
                             values_from = podatek)
```

	rok	kraj	podatek
1	1900	USA	0.00000000
2	1900	WB	0.00000000
3	1900	Niemcy	0.03000000
4	1900	Francja	0.00000000
5	1901	USA	0.00000000
6	1901	WB	0.00000000
7	1901	Niemcy	0.03000000
8	1901	Francja	0.00000000

Showing 1 to 8 of 456 entries, 3 total

	rok	USA	WB	Niemcy	Francja
1	1900	0.0000	0.00000000	0.03	0.000000
2	1901	0.0000	0.00000000	0.03	0.000000
3	1902	0.0000	0.00000000	0.03	0.000000
4	1903	0.0000	0.00000000	0.03	0.000000
5	1904	0.0000	0.00000000	0.03	0.000000
6	1905	0.0000	0.00000000	0.03	0.000000
7	1906	0.0000	0.00000000	0.03	0.000000
8	1907	0.0000	0.00000000	0.03	0.000000

Showing 1 to 8 of 114 entries, 5 total columns

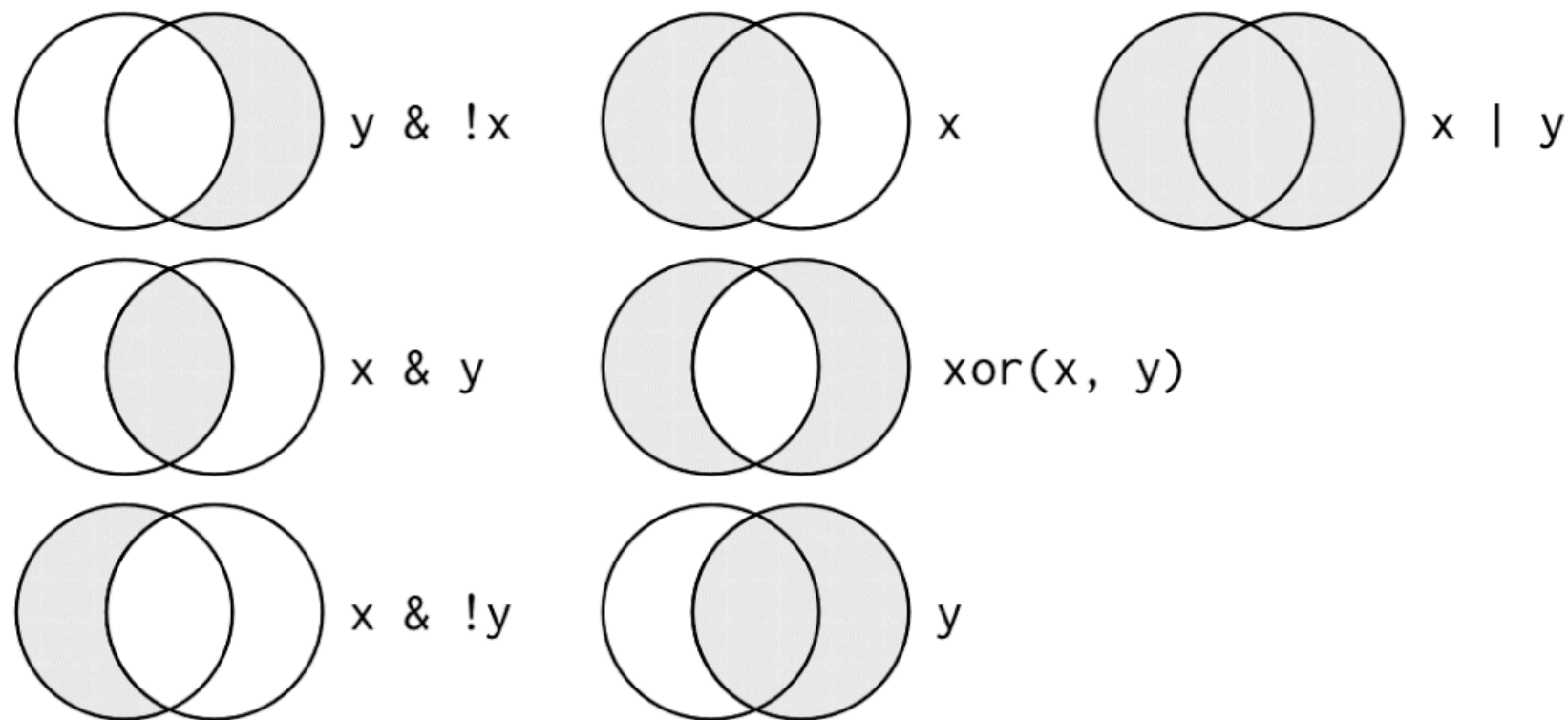
Podstawowe funkcje DPLYR

- Wybieranie obserwacji na podstawie ich wartości: **filter()**.
- Zmiana kolejności wierszy: **arrange()**.
- Wybieranie zmiennych na podstawie ich nazw: **select()**.
 - Tworzenie nowych zmiennych za pomocą funkcji istniejących zmiennych: **mutate()**.
- Zwijanie wielu wartości do podsumowania: **summarize()**.

Operatory logiczne w R

Operator	Description
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
==	Equal to
!=	Not equal to
!x	Not x
x y	x OR y
x & y	x AND y

działanie operatorów logicznych



Funkcje pomocnicze funkcji select()

- starts_with(„abc”) wybiera nazwy rozpoczynające się od „abc”.
- ends_with(„xyz”) wybiera nazwy kończące się na „xyz”
- contains(„ijk”) wybiera nazwy zawierające „ijk”
- matches() wybiera zmienne pasujące do wyrażenia regularnego
- num_range

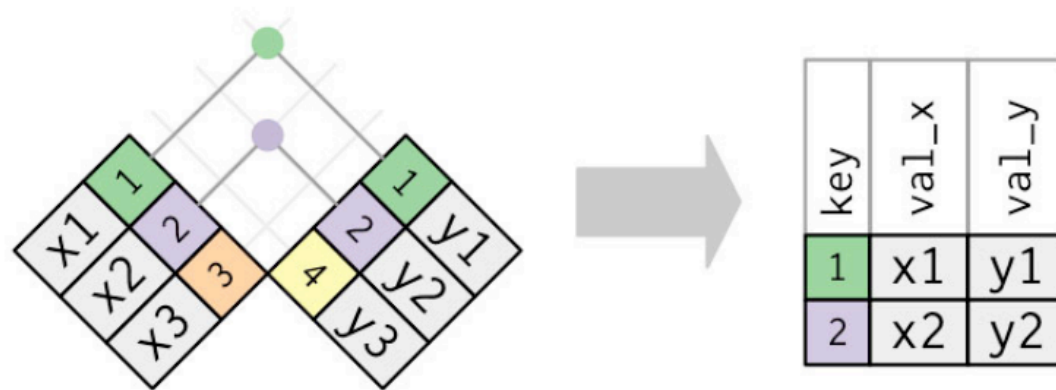
Klucze

zmienne służące do łączenia pary tablic

- klucz główny
- klucz obcy

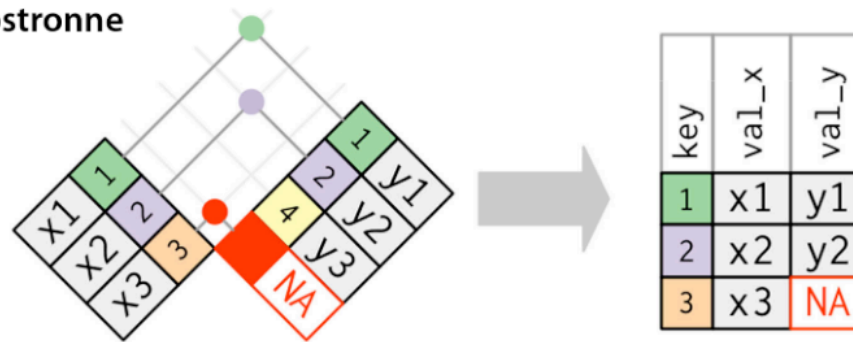
Złączenie wewnętrzne

inner_join()

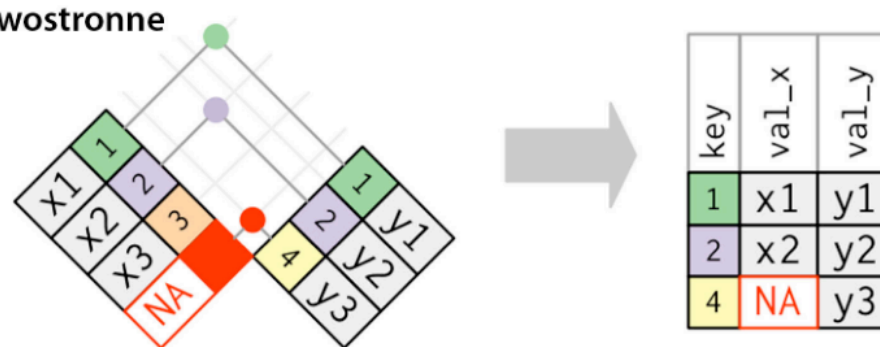


złączenia zewnętrzne

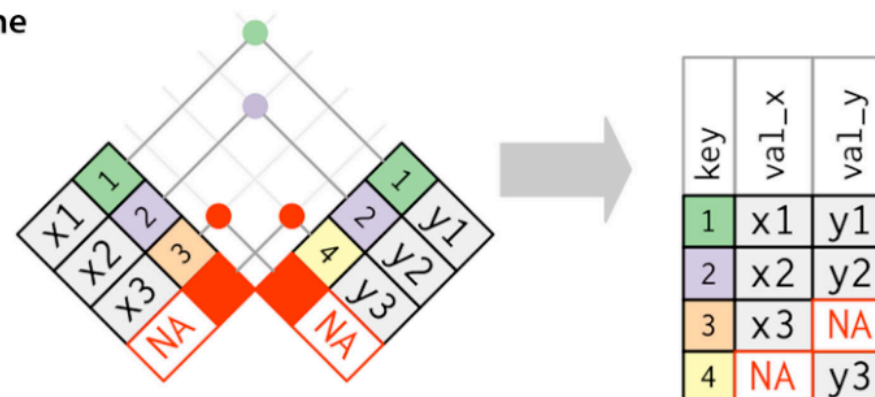
Lewostronne

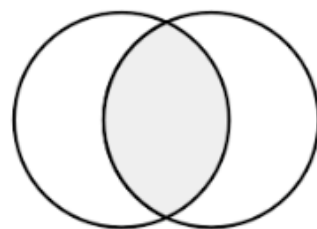


Prawostronne

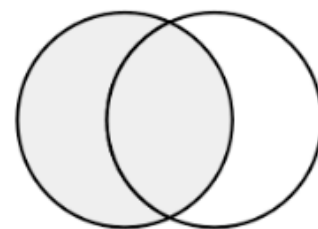


Pełne

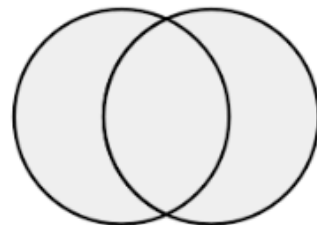




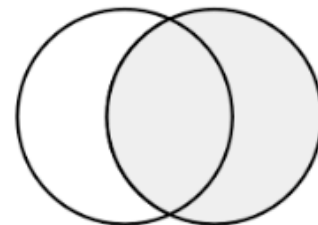
inner_join(x, y)



left_join(x, y)



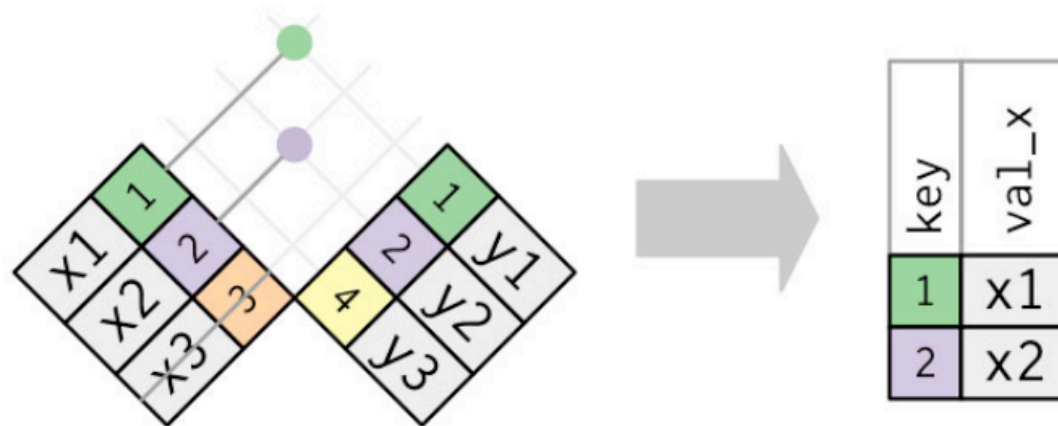
full_join(x, y)



right_join(x, y)

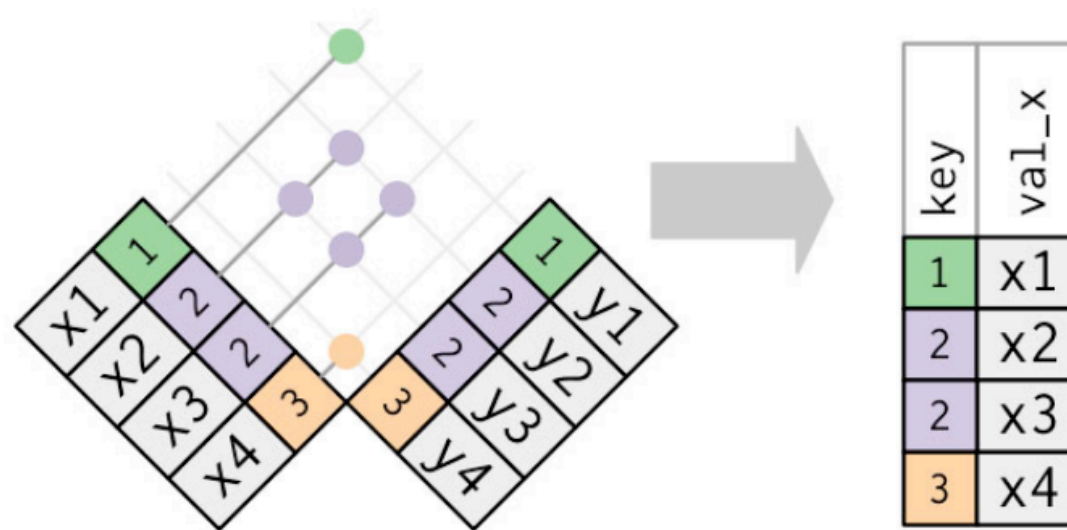
Złączenie częściowe

`semi_join()`



antyłączenia

anti_join()



dplyr	merge
<code>inner_join(x, y)</code>	<code>merge(x, y)</code>
<code>left_join(x,y)</code>	<code>merge(x,y,all.x=TRUE)</code>
<code>right_join(x, y)</code>	<code>merge(x, y, all.y = TRUE)</code>
<code>full_join(x,y)</code>	<code>merge(x,y,all.x=TRUE,all.y=TRUE)</code>

dplyr	SQL
<code>inner_join(x, y, by = "z")</code>	<code>SELECT * FROM x INNER JOIN y USING (z)</code>
<code>left_join(x,y,by="z")</code>	<code>SELECT * FROM x LEFT OUTER JOIN y USING (z)</code>
<code>right_join(x, y, by = "z")</code>	<code>SELECT * FROM x RIGHT OUTER JOIN y USING (z)</code>
<code>full_join(x,y,by="z")</code>	<code>SELECT * FROM x FULL OUTER JOIN y USING (z)</code>